In the criteria it is said that the marker should be able to run the code from only checkers.py. It was very inconvenient to work on complex tasks such as AI for checkers in a single file.

I have faced the problem that successors function worked at small scale, but when implemented to the whole project, it produced infinite loops with the same successors. It turned out that I forgot how my own code operated. While fixing the problem of invalid moves, I slightly changed the code. I found a mistake that my code produced successors only to small number of checkers. I thought that new_successors creates successors only to one checker and sends it to computeSuccessors but I forgot that new_successors takes successors list and sends it back. Because of that my "fix" made new_successors produce list of successors and added to the same list, while it was supposed to add one successor and give back the list.

After that I encountered new problem: successors function returns only the first available successor if there is an enemy to eat and it does not consider possibility for multiple edible enemies to exist. It happened because successors function consists of two functions. The first was going through every position in the desk and sending the positions of pieces available to move, direction of moving and current successors to the seconds function, which was calculating successors and sending them back.. Two functions were just sending the same successors between each other. After trying to fix problem with multiple edible enemies I discovered that it much better for the first function to send only the current desk, position of the piece and direction to move. The second function would return the updated desk and boolean value representing whether the piece have eaten the enemy or not. If piece have not eaten anyone the desk would be appended to the list with basic successors. If piece have eaten someone the desk would be added to the list of eat successors. At the end if eat successors is not empty function will return eat successors list, otherwise it will send basic successors list.

I successfully added multi-step capturing moves for AI, but could not do that for player. For AI I only needed to add additional function to add capture successors to the eat_successors list. But for player I could not separate additional capturing with just additional steps. If player captured enemy he could also make one move without capturing. I could not find a way around it. It has also led to different problem. The algorithm computes the same successors for AI and player, meaning that when AI computes possible player movements it takes into account players ability to capture multiple figures. However this ability does not exist for the player.

I tried creating the board as shown in checkers rules, but it was painful for my eyes, so I changed colors to more pleasant.

The change in difficulty is realised by changing the maximum depth for minimax with alpha beta pruning.

Hard difficulty AI takes a lot of time to make move if there is no openly disadvantageous move for player. It happens closer to the late stages of the game. That's why I added a time constraint of 20 seconds.